// TUTORIAL //

How To Use LVM To Manage Storage Devices on Ubuntu 18.04

Updated on January 5, 2023

Linux Basics Ubuntu System Tools Storage Ubuntu 18.04

Justin Ellingwood

Not using Ubuntu 18.04?
Choose a different version or distribution.

Ubuntu 18.04

Introduction

Logical Volume Management, or *LVM*, is a storage device management technology that gives users the power to pool and abstract the physical layout of component

storage devices for flexible administration. Using the device mapper Linux kernel framework, the current iteration, LVM2, can be used to gather existing storage devices into groups and allocate logical units from the combined space as needed.

In this tutorial, you'll learn how to manage LVM by displaying information about volumes and potential targets, create and destroy volumes of various types, and modify existing volumes through resizing or transformation.

Prerequisites

To follow along, you will need to have a non-**root** user with `sudo` privileges configured on an Ubuntu 18.04 server. You can follow our Ubuntu 18.04 Initial Server Setup guide to get started.

Also, if you're not familiar with LVM components and concepts, you can review our Introduction to LVM guide for more information.

When you are ready, log into your server with your `sudo` user.

Step 1 – Displaying Information About Physical Volumes, Volume Groups, and Logical Volumes

Accessing information about the various LVM components on your system is essential for managing your physical and logical volumes. LVM provides a number of tools for displaying information about every layer in the LVM stack.

Displaying Information About All LVM Compatible Block Storage Devices

To display all of the available block storage devices that LVM can potentially manage, use the `lvmdiskscan` command:

```
$ sudo lvmdiskscan
```

```
Output
   /dev/sda   [     200.00 GiB]
   /dev/sdb   [     100.00 GiB]
   2 disks
  2 partitions
```

```
  0 LVM physical volume whole disks
  0 LVM physical volumes
```

Notice the devices that can potentially be used as physical volumes for LVM.

This will likely be your first step when adding new storage devices to use with LVM.

Displaying Information about Physical Volumes

A header is written to storage devices to mark them as free to use as LVM components. Devices with these headers are called *physical volumes*.

You can display all of the physical devices on your system by using `lvmdiskscan` with the `-l` option, which will only return physical volumes:

```
$ sudo lvmdiskscan -l
```

```
Output
  WARNING: only considering LVM devices
  /dev/sda                  [     200.00 GiB] LVM physical volume
  /dev/sdb                  [     100.00 GiB] LVM physical volume
  2 LVM physical volume whole disks
  0 LVM physical volumes
```

The `pvscan` command is similar in that it searches all available devices for LVM physical volumes. The output format includes a small amount of additional information:

```
$ sudo pvscan
```

```
Output
  PV /dev/sda   VG LVMVolGroup     lvm2 [200.00 GiB / 0    free]
  PV /dev/sdb   VG LVMVolGroup     lvm2 [100.00 GiB / 10.00 GiB free]
  Total: 2 [299.99 GiB] / in use: 2 [299.99 GiB] / in no VG: 0 [0   ]
```

If you need additional details about your volume, the `pvs` and `pvdisplay` commands

can do that for you.

The `pvs` command is highly configurable and can display information in many different formats. Because its output can be tightly controlled, it is frequently used when scripting or automation is needed. Its basic output provides a useful at-a-glance summary similar to the earlier commands:

```
$ sudo pvs
```

```
Output
  PV         VG          Fmt  Attr PSize   PFree
  /dev/sda   LVMVolGroup lvm2 a--  200.00g     0
  /dev/sdb   LVMVolGroup lvm2 a--  100.00g 10.00g
```

For more verbose, human-readable output, the `pvdisplay` command is a good option:

```
$ sudo pvdisplay
```

```
Output
  --- Physical volume ---
  PV Name               /dev/sda
  VG Name               LVMVolGroup
  PV Size               200.00 GiB / not usable 4.00 MiB
  Allocatable           yes (but full)
  PE Size               4.00 MiB
  Total PE              51199
  Free PE               0
  Allocated PE          51199
  PV UUID               kRUOyU-0ib4-ujPh-kAJP-eeQv-ztRL-4EkaDQ

  --- Physical volume ---
  PV Name               /dev/sdb
  VG Name               LVMVolGroup
  PV Size               100.00 GiB / not usable 4.00 MiB
  Allocatable           yes
  PE Size               4.00 MiB
  Total PE              25599
  Free PE               2560
```

```
  Allocated PE          23039
  PV UUID               udcuRJ-jCDC-26nD-ro9u-QQNd-D6VL-GEIlD7
```

To discover the logical extents that have been mapped to each volume, pass in the `-m` option to `pvdisplay`:

```
$ sudo pvdisplay -m
```

```
Output
  --- Physical volume ---
  PV Name               /dev/sda
  VG Name               LVMVolGroup
  PV Size               200.00 GiB / not usable 4.00 MiB
  Allocatable           yes
  PE Size               4.00 MiB
  Total PE              51199
  Free PE               38395
  Allocated PE          12804
  PV UUID               kRUOyU-0ib4-ujPh-kAJP-eeQv-ztRL-4EkaDQ

  --- Physical Segments ---
  Physical extent 0 to 0:
    Logical volume  /dev/LVMVolGroup/db_rmeta_0
    Logical extents 0 to 0
  Physical extent 1 to 5120:
    Logical volume  /dev/LVMVolGroup/db_rimage_0
    Logical extents 0 to 5119

   . . .
```

This can be very useful when trying to determine which data is held on which physical disk for management purposes.

Displaying Information about Volume Groups

LVM also has plenty of tools to display information about volume groups.

The `vgscan` command can be used to scan the system for available volume groups. It also rebuilds the cache file when necessary. It is a good command to use when you are importing a volume group into a new system:

```
$ sudo vgscan
```

```
Output
  Reading all physical volumes.  This may take a while...
  Found volume group "LVMVolGroup" using metadata type lvm2
```

This command does not output very much information, but it should be able to find every available volume group on the system. To display more information, the `vgs` and `vgdisplay` commands are available.

Like its physical volume counterpart, the `vgs` command is versatile and can display a large amount of information in a variety of formats. Because its output can be manipulated, it is frequently used when scripting or automation is needed. For example, some helpful output modifications are to show the physical devices and the logical volume path:

```
$ sudo vgs -o +devices,lv_path
```

```
Output
  VG          #PV #LV #SN Attr   VSize   VFree  Devices        Path
  LVMVolGroup   2   4   0 wz--n- 299.99g 10.00g /dev/sda(0)    /dev/LVMVolG
  LVMVolGroup   2   4   0 wz--n- 299.99g 10.00g /dev/sda(2560) /dev/LVMVolG
  LVMVolGroup   2   4   0 wz--n- 299.99g 10.00g /dev/sda(3840) /dev/LVMVolG
  LVMVolGroup   2   4   0 wz--n- 299.99g 10.00g /dev/sda(8960) /dev/LVMVolG
  LVMVolGroup   2   4   0 wz--n- 299.99g 10.00g /dev/sdb(0)    /dev/LVMVolG
```

Likewise, for more verbose, human-readable output, use the `vgdisplay` command. Adding the `-v` flag provides information about the physical volumes the volume group is built upon, and the logical volumes that were created using the volume group:

```
$ sudo vgdisplay -v
```

```
Output
    Using volume group(s) on command line.
  --- Volume group ---
  VG Name               LVMVolGroup
  . . .

  --- Logical volume ---
  LV Path                /dev/LVMVolGroup/projects
  . . .

  --- Logical volume ---
  LV Path                /dev/LVMVolGroup/www
  . . .

  --- Logical volume ---
  LV Path                /dev/LVMVolGroup/db
  . . .

  --- Logical volume ---
  LV Path                /dev/LVMVolGroup/workspace
  . . .

  --- Physical volumes ---
  PV Name               /dev/sda
  . . .

  PV Name               /dev/sdb
  . . .
```

The `vgdisplay` command is useful because it can tie together information about many different elements of the LVM stack.

Displaying Information about Logical Volumes

To display information about logical volumes, LVM has a related set of tools.

As with the other LVM components, the `lvscan` option scans the system and outputs minimal information about the logical volumes it finds:

```
$ sudo lvscan
```

```
Output
```

```
  ACTIVE            '/dev/LVMVolGroup/projects' [10.00 GiB] inherit
  ACTIVE            '/dev/LVMVolGroup/www' [5.00 GiB] inherit
  ACTIVE            '/dev/LVMVolGroup/db' [20.00 GiB] inherit
  ACTIVE            '/dev/LVMVolGroup/workspace' [254.99 GiB] inherit
```

For more complete information, the `lvs` command is flexible and powerful to use in scripts:

```
$ sudo lvs
```

```
Output
  LV        VG          Attr       LSize   Pool Origin Data%  Meta%  Move L
  db        LVMVolGroup -wi-ao----  20.00g
  projects  LVMVolGroup -wi-ao----  10.00g
  workspace LVMVolGroup -wi-ao---- 254.99g
  www       LVMVolGroup -wi-ao----   5.00g
```

To find the number of stripes and the logical volume type, use the `--segments` option:

```
$ sudo lvs --segments
```

```
Output
  LV           VG          Attr       #Str Type    SSize
  db           LVMVolGroup rwi-a-r---    2 raid1   20.00g
  mirrored_vol LVMVolGroup rwi-a-r---    3 raid1   10.00g
  test         LVMVolGroup rwi-a-r---    3 raid5   10.00g
  test2        LVMVolGroup -wi-a-----    2 striped 10.00g
  test3        LVMVolGroup rwi-a-r---    2 raid1   10.00g
```

The most human-readable output is produced by the `lvdisplay` command.

When the `-m` flag is added, the tool will also display information about how the logical volume is broken down and distributed:

```
$ sudo lvdisplay -m
```

```
Output
  --- Logical volume ---
  LV Path                /dev/LVMVolGroup/projects
  LV Name                projects
  VG Name                LVMVolGroup
  LV UUID                IN4GZm-ePJU-zAAn-DRO3-1f2w-qSN8-ahisNK
  LV Write Access        read/write
  LV Creation host, time lvmtest, 2016-09-09 21:00:03 +0000
  LV Status              available
  # open                 1
  LV Size                10.00 GiB
  Current LE             2560
  Segments               1
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           252:0

  --- Segments ---
  Logical extents 0 to 2559:
    Type        linear
    Physical volume /dev/sda
    Physical extents    0 to 2559


  . . .
```

In this example, the `/dev/LVMVolGroup/projects` logical volume is contained entirely within the `/dev/sda` physical volume. This information is useful if you need to remove that underlying device and wish to move the data off to specific locations.

Step 2 – Creating or Extending LVM Components

This section discusses how to create and expand physical volumes, volume groups, and logical volumes.

Creating Physical Volumes From Raw Storage Devices

To use storage devices with LVM, they must first be marked as a physical volume. This specifies that LVM can use the device within a volume group.

First, use the `lvmdiskscan` command to find all block devices that LVM can access and use:

```
$ sudo lvmdiskscan
```

```
Output
   /dev/sda   [     200.00 GiB]
   /dev/sdb   [     100.00 GiB]
   2 disks
  2 partitions
  0 LVM physical volume whole disks
  0 LVM physical volumes
```

Here, notice the devices that are suitable to be turned into physical volumes for LVM.

> **Warning**: Make sure that you double-check that the devices you intend to use with LVM do not have any important data already written to them. Using these devices within LVM will overwrite the current contents. If you have important data on your server, make backups before proceeding.

To mark the storage devices as LVM physical volumes, use `pvcreate`. You can pass in multiple devices at once:

```
$ sudo pvcreate /dev/ sda  /dev/ sdb
```

This command writes an LVM header on all of the target devices to mark them as LVM physical volumes.

Creating a New Volume Group from Physical Volumes

To create a new volume group from LVM physical volumes, use the `vgcreate` command. You have to provide a volume group name, followed by at least one LVM physical volume:

```
$ sudo vgcreate  volume_group_name  /dev/ sda
```

This example creates your volume group with a single initial physical volume. You can pass in more than one physical volume at creation if you'd like:

```
$ sudo vgcreate  volume_group_name  /dev/ sda  /dev/ sdb  /dev/ sdc
```

Usually, you only need a single volume group per server. All LVM-managed storage can be added to that pool and then logical volumes can be allocated from that.

One reason you may wish to have more than one volume group is if you feel you need to use different extent sizes for different volumes. You don't typically have to set the extent size (the default size of 4M is adequate for most uses), but if you need to, you can do so upon volume group creation by passing the `-s` option:

```
$ suod vgcreate -s  8M   volume_group_name  /dev/ sda
```

This will create a new volume group with an 8M extent size.

Adding a Physical Volume to an Existing Volume Group

To expand a volume group by adding additional physical volumes, use the `vgextend` command. This command takes a volume group followed by the physical volumes to add. You can pass in multiple devices at once if you'd like:

```
$ sudo vgextend  volume_group_name  /dev/ sdb
```

The physical volume will be added to the volume group, expanding the available capacity of the storage pool.

Creating a Logical Volume by Specifying Size

To create a logical volume from a volume group storage pool, use the `lvcreate` command. Specify the size of the logical volume with the `-L` option, then specify a name with the `-n` option, and pass in the volume group to allocate the space from.

For instance, to create a 10G logical volume named `test` from the `LVMVolGroup` volume group, write:

```
$ sudo lvcreate -L 10G -n test LVMVolGroup
```

If the volume group has enough free space to accommodate the volume capacity, the new logical volume will be created.

Creating a Logical Volume From All Remaining Free Space

If you wish to create a volume using the remaining free space within a volume group, use the `vgcreate` command with the `-n` option to name and pass in the volume group like the previous step. Instead of passing in a size, use the `-l 100%FREE` option, which uses the remaining extents within the volume group to form the logical volume:

```
$ sudo lvcreate -l 100%FREE -n test2 LVMVolGroup
```

This should use up the remaining space in the logical volume.

Creating Logical Volumes with Advanced Options

Logical volumes can be created with some advanced options. Some options that you may wish to consider are:

- `--type`: This specifies the type of logical volume, which determines how the logical volume is allocated. Some available types will not be available if there are not enough underlying physical volumes to correctly create the chosen topography. Some of the most common types are:
  - `linear`: The default type. The underlying physical devices used, if more than one, will be appended to each other, one after the other.
  - `striped`: Similar to RAID 0, the striped topology divides data into chunks and spread in a round-robin fashion across the underlying physical volumes. This can lead to performance improvements, but might lead to greater data vulnerability. This requires the `-i` option and a minimum of two physical volumes.
  - `raid1`: Creates a mirrored RAID 1 volume. By default, the mirror will have two copies, but more can be specified by the `-m`. This requires a minimum

of two physical volumes.
- `raid5`: Creates a RAID 5 volume. This requires a minimum of three physical volumes.
- `raid6`: Creates a RAID 6 volume. This requires a minimum of four physical volumes.

- `-m`: Specifies the number of additional copies of data to keep. A value of "1" specifies that one additional copy is maintained, for a total of two sets of data.
- `-i`: Specifies the number of stripes that should be maintained. This is required for the `striped` type, and can modify the default behavior of some of the other RAID options.
- `-s`: Specifies that the action should create a snapshot from an existing logical volume instead of a new independent logical volume.

To demonstrate, begin by creating a striped volume. You must specify at least two stripes for this method. This topology and stripe count requires a minimum of two physical volumes with available capacity:

```
$ sudo lvcreate --type striped -i 2 -L 10G -n striped_vol LVMVolGroup
```

To create a mirrored volume, use the `raid1` type. If you want more than two sets of data, use the `-m` option. This example uses `-m 2` to create a total of three sets of data. LVM counts this as one original data set with two mirrors. You need at least three physical volumes for this to succeed:

```
$ sudo lvcreate --type raid1 -m 2 -L 20G -n mirrored_vol LVMVolGroup
```

To create a snapshot of a volume, you must provide the original logical volume to snapshot instead of the volume group. Snapshots do not take up much space initially, but grow in size as changes are made to the logical volume it is tracking. The size used during this procedure is the maximum size that the snapshot can be. Snapshots that grow past this size are broken and cannot be used, however, snapshots approaching their capacity can be extended:

```
$ sudo lvcreate -s -L 10G -n  snap_test   LVMVolGroup / test
```

Note: To revert a logical volume to the point-in-time of a snapshot, use the `lvconvert --merge` command:

```
$ sudo lvconvert --merge  LVMVolGroup / snap_test
```

This will bring the origin of the snapshot back to the state when the snapshot was taken.

There are a number of options that can dramatically alter the way that your logical volumes function.

Growing the Size of a Logical Volume

One of the main advantages of LVM is the flexibility it provides in provisioning logical volumes. You can adjust the number or size of volumes on the fly without stopping the system.

To grow the size of an existing logical volume, use the `lvresize` command. Use the `-L` flag to specify a new size. You can also use relative sizes by adding a `+` size. In that case, LVM will increase the size of the logical volume by the amount specified. To automatically resize the filesystem being used on the logical volume, pass in the `--resizefs` flag.

To correctly provide the name of the logical volume to expand, you need to give the volume group, followed by a slash, followed by the logical volume:

```
$ sudo lvresize -L  +5G  --resizefs  LVMVolGroup / test
```

In this example, the logical volume and the filesystem of the `test` logical volume on the `LVMVolGroup` volume group will both be increased by 5G.

If you wish to handle the filesystem expansion manually, take out the `--resizefs` option and use the filesystem's native expansion utility afterwards. For example, for

an Ext4 filesystem, write:

```
$ sudo lvresize -L  +5G   LVMVolGroup / test
$ sudo resize2fs /dev/ LVMVolGroup / test
```

This returns the same result.

Step 3 – Removing or Downsizing LVM Components

Since capacity reduction can result in data loss, the procedures to shrink the available capacity, either by reducing the size of or removing components are typically a bit more involved.

Reducing the Size of a Logical Volume

To shrink a logical volume, you should first back up your data. Because this reduces the available capacity, mistakes can lead to data loss.

When you are ready, check on how much space is currently being used:

```
$ df -h
```

```
Output
Filesystem                          Size  Used Avail Use% Mounted on
. . .
/dev/mapper/LVMVolGroup-test        4.8G  521M  4.1G  12% /mnt/test
```

In this example, a little over 521M of the space is currently in use. Use this to help you estimate the size that you can reduce the volume to.

Unlike expansions, filesystem shrinking should be performed when unmounted. First, make sure you're in the root directory:

```
$ cd ~
```

Next, unmount the filesystem:

```
$ sudo umount /dev/ LVMVolGroup / test
```

After unmounting, check the filesystem to ensure that everything is in working order. Pass in the filesystem type with the `-t` option. Use `-f` to check when the filesystem appears:

```
$ sudo fsck -t  ext4  -f /dev/ LVMVolGroup / test
```

After checking the filesystem, you can reduce the filesystem size using the filesystem's native tools. For Ext4 filesystems, this would be the `resize2fs` command. Pass in the final size for the filesystem:

> **Warning**: The safest option here is to choose a final size that is a fair amount larger than your current usage. Give yourself some buffer room to avoid data loss and ensure that you have backups in place.

```
$ sudo resize2fs -p /dev/ LVMVolGroup / test   3G
```

Once the operation is complete, resize the logical volume by passing the same size to the `lvresize` command with the `-L` flag:

```
$ sudo lvresize -L  3G   LVMVolGroup / test
```

You are warned about the possibility of data loss. If you are ready, enter `y` to

proceed.

After the logical volume has been reduced, check the filesystem again:

```
$ sudo fsck -t ext4 -f /dev/ LVMVolGroup / test
```

If everything is functioning correctly, you can remount the filesystem using your usual mount command:

```
$ sudo mount /dev/ LVMVolGroup / test  /mnt/ test
```

Your logical volume should now be reduced to the appropriate size.

Removing a Logical Volume

If you no longer need a logical volume, you can remove it with the `lvremove` command.

First, unmount the logical volume if it is currently mounted:

```
$ cd ~
$ sudo umount /dev/ LVMVolGroup / test
```

Afterwards, remove the logical volume by entering this command:

```
$ sudo lvremove  LVMVolGroup / test
```

You are asked to confirm the procedure. If you are certain you want to delete the logical volume, press `y`.

Removing a Volume Group

To remove an entire volume group, including all of the logical volumes within it, use the `vgremove` command.

Before you remove a volume group, you should remove the logical volumes using the procedure previously discussed. At the very least, you must make sure that you unmount any logical volumes that the volume group contains:

```
$ sudo umount /dev/ LVMVolGroup / www
$ sudo umount /dev/ LVMVolGroup / projects
$ sudo umount /dev/ LVMVolGroup / db
```

Afterwards, you can delete the entire volume group by passing the volume group name to the `vgremove` command:

```
$ sudo vgremove  LVMVolGroup
```

You are then prompted to confirm that you wish to remove the volume group. If you have any logical volumes still present, you are given individual confirmation prompts for those before removing.

Removing a Physical Volume

To remove a physical volume from LVM management, the procedure you need depends on whether the device is currently being used by LVM.

If the physical volume is in use, you have to move the physical extents located on the device to a different location. This requires the volume group to have enough other physical volumes to handle the physical extents. If you are using more complex logical volume types, you might need additional physical volumes even when you have plenty of free space to accommodate the topology.

When you have enough physical volumes in the volume group to handle the physical extents, move them off of the physical volume you wish to remove by running:

```
$ sudo pvmove /dev/ sda
```

This process can take time depending on the size of the volumes and the amount of data to transfer.

Once the extents have been relocated to peer volumes, you can remove the physical volume from the volume group:

```
$ sudo vgreduce  LVMVolGroup  /dev/ sda
```

This removes the vacated physical volume from the volume group. After this is complete, you can remove the physical volume marker from the storage device:

```
$ sudo pvremove /dev/ sda
```

You can now use the removed storage device for other purposes or remove it from the system entirely.

Conclusion

You now have an understanding of how to manage storage devices on Ubuntu 18.04 with LVM. You also know how to get information about the state of existing LVM components, how to use LVM to compose your storage system, and how to modify volumes to meet your needs. Feel free to test these concepts in a safe environment to get a better grasp of how they fit together.